

Introducción y fundamentos estadísticos

Clase 7 · Evaluar y regularizar modelos

Daniela Opitz

Diplomado en Ciencia de Datos Aplicada · Departamento de Informática · UTFSM · martes 15 de septiembre, 2026



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

DEPARTAMENTO
DE INFORMÁTICA

Avisos del proyecto Capstone

- ▶ **Hoy, martes 15 de septiembre, se entrega el primer análisis exploratorio por el Aula.**
- ▶ Informe de avance, la última evaluación: presentación oral el martes 22 en la clase 8 (máximo 10 minutos por equipo) e informe escrito el viernes 25 (máximo 7 páginas). Oral y escrito pesan 50% cada uno; el enunciado está en el repositorio.
- ▶ El informe de avance debe incluir un modelo ajustado a los datos del proyecto: uno de los vistos en clases u otro que el equipo ya maneje.
- ▶ El jueves 17 no hay clases.

Contenidos de la clase

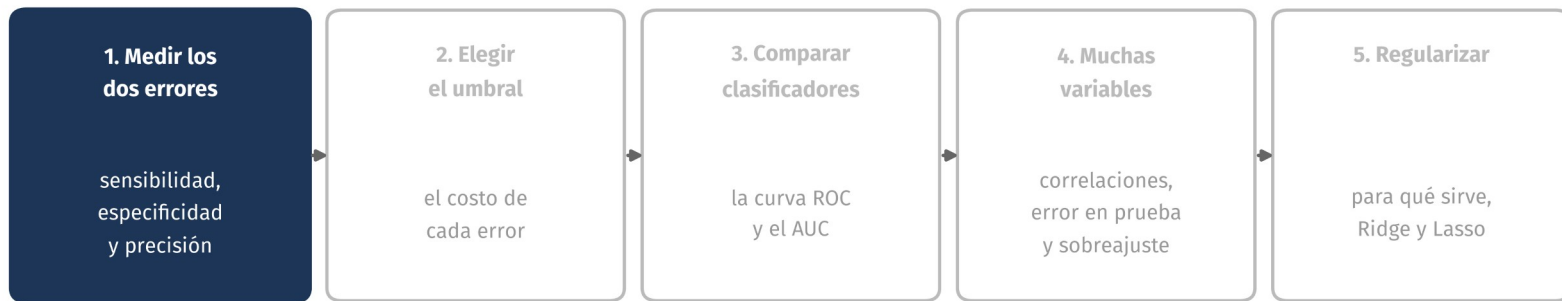
- ▶ 1. Medir los dos errores de un clasificador: sensibilidad, especificidad y precisión.
- ▶ 2. Elegir el umbral según el costo de cada error.
- ▶ 3. Comparar clasificadores: la curva ROC y el AUC.
- ▶ 4. Muchas variables: qué dicen las correlaciones, el error en datos de prueba y el sobreajuste.
- ▶ 5. Regularizar: para qué sirve, Ridge y Lasso, y cómo elegir cuánto castigar.

¿Cuánto cuesta equivocarse?

- ▶ **La clase 6 dejó dos preguntas abiertas: si con un sí raro la exactitud engaña, ¿qué se mide? Y el umbral 0,5, ¿por qué 0,5?**
- ▶ La primera mitad las responde: medidas que miran cada error por separado, y un umbral que sale del costo de cada error.
- ▶ La segunda mitad vuelve a la regresión: con muchas variables, el modelo se ajusta al azar de su muestra y se equivoca más en datos nuevos. Regularizar lo corrige.
- ▶ El hilo de las dos: un modelo se juzga por sus errores donde importan, con el costo correcto y en datos que no vio.

El mapa de la clase

Un modelo se juzga por sus errores donde importan: con el costo correcto y en datos que no vio



clasificar: los clientes de la tarjeta

regresión: los salarios de béisbol

Recordatorio: la matriz de confusión

La matriz de confusión de la clase 6: umbral 0,5, 10.000 clientes		
	predicho "no paga"	predicho "paga"
real: no paga	Verdadero positivo 100 el modelo acertó	Falso negativo 233 era sí, el modelo dijo no
real: paga	Falso positivo 42 era no, el modelo dijo sí	Verdadero negativo 9.625 el modelo acertó

Lo real en las filas, lo predicho en las columnas. Con umbral 0,5 acierta el 97,3%; decir que todos pagan, el 96,7%: la exactitud mezcla los dos errores. Aquí no se parte en entrenamiento y prueba: con una variable y 10.000 clientes da lo mismo (clase 6); con muchas variables no, y eso viene en la segunda mitad.

Sensibilidad: ¿cuántos morosos detecta?

	predicho "no paga"	predicho "paga"
real: no paga	Verdadero positivo 100 el modelo acertó	Falso negativo 233 era sí, el modelo dijo no
real: paga	Falso positivo 42 era no, el modelo dijo sí	Verdadero negativo 9.625 el modelo acertó

Sensibilidad

De los clientes que no pagan, ¿qué fracción detecta?

Se mira la fila de los que no pagan (en inglés, recall).

$$\text{sensibilidad} = \text{VP} / (\text{VP} + \text{FN})$$

$$= 100 / (100 + 233) = 100 / 333 = 0,30$$

El modelo detecta 3 de cada 10 clientes que no pagan; se le escapan 7.

Mira solo a los clientes que no pagan. Es la medida que importa cuando dejar pasar un sí es caro: un moroso sin detectar, una enfermedad sin diagnosticar.

Especificidad: ¿a cuántos deja en paz?

	predicho "no paga"	predicho "paga"
real: no paga	Verdadero positivo 100 el modelo acertó	Falso negativo 233 era sí, el modelo dijo no
real: paga	Falso positivo 42 era no, el modelo dijo sí	Verdadero negativo 9.625 el modelo acertó

Especificidad

De los clientes que pagan, ¿qué fracción deja en paz?

Se mira la fila de los que pagan.

$$\text{especificidad} = \text{VN} / (\text{VN} + \text{FP})$$

$$= 9.625 / (9.625 + 42) = 0,996$$

El modelo casi no molesta a los que pagan:
de cada 1.000, a 996 no los declara morosos.

Mira solo a los clientes que pagan. Importa cuando molestar a un no es caro: negarle la tarjeta a un buen cliente, alarmar a un paciente sano.

Precisión: de los que marca, ¿cuántos lo son?

	predicho "no paga"	predicho "paga"
real: no paga	Verdadero positivo 100 el modelo acertó	Falso negativo 233 era sí, el modelo dijo no
real: paga	Falso positivo 42 era no, el modelo dijo sí	Verdadero negativo 9.625 el modelo acertó

Precisión

De los clientes que declara morosos, ¿qué fracción lo es?

Se mira la columna del predicho "no paga" (en inglés, precision).

precisión = $VP / (VP + FP)$

= $100 / (100 + 42) = 100 / 142 = 0,70$

De cada 10 clientes que el modelo declara morosos, 7 lo son de verdad.

Mira solo a los clientes que el modelo declara morosos. Importa cuando actuar sobre cada sí cuesta: en el libro, una aseguradora que llama solo a los clientes que el modelo marca logra que compre el 33% de los llamados, contra el 6% llamando al azar (James et al., pp. 165 a 167).

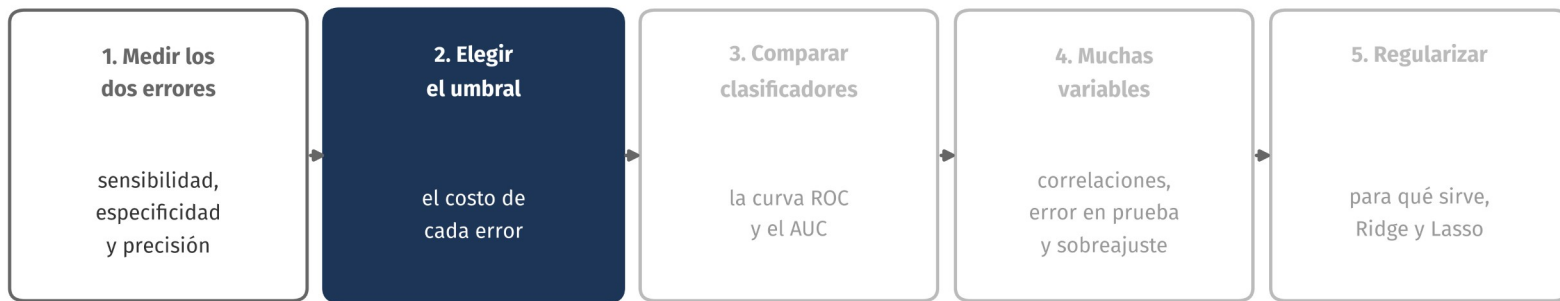
Cuatro medidas, una sola matriz

medida	la pregunta que responde	fórmula	qué parte mira	umbral 0,5
Exactitud (accuracy)	De todos, ¿qué fracción acertó?	$(VP + VN) / \text{total}$	toda la matriz	0,973
Sensibilidad (recall)	De los que no pagan, ¿qué fracción detecta?	$VP / (VP + FN)$	la fila del sí	0,30
Especificidad	De los que pagan, ¿qué fracción deja en paz?	$VN / (VN + FP)$	la fila del no	0,996
Precisión (precision)	De los que declara morosos, ¿qué fracción lo es?	$VP / (VP + FP)$	la columna del sí	0,70

Todas salen de contar las celdas de la matriz. La exactitud mira todo junto; las otras tres miran un error por vez, y por eso no engañan cuando el sí es raro.

El mapa de la clase

Un modelo se juzga por sus errores donde importan: con el costo correcto y en datos que no vio



clasificar: los clientes de la tarjeta

regresión: los salarios de béisbol

Mover el umbral cambia los errores

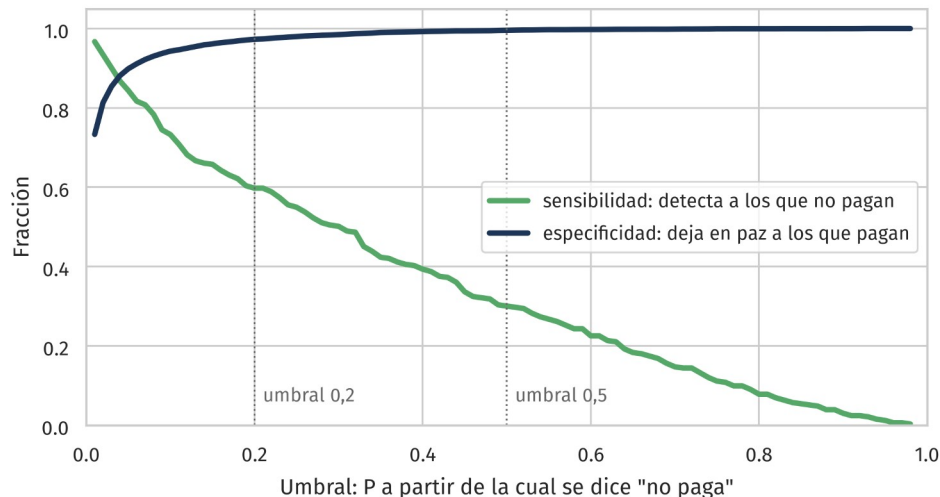
Bajar el umbral a 0,2: el modelo detecta 99 clientes más que no pagan, a cambio de 221 falsos positivos más

Umbral 0,5				Umbral 0,2			
predicho "no paga"		predicho "paga"		predicho "no paga"		predicho "paga"	
real: no paga	Verdadero positivo 100 el modelo acertó	Falso negativo 233 era sí, el modelo dijo no		real: no paga	Verdadero positivo 199 el modelo acertó	Falso negativo 134 era sí, el modelo dijo no	
	Falso positivo 42 era no, el modelo dijo sí	Verdadero negativo 9.625 el modelo acertó			Falso positivo 263 era no, el modelo dijo sí	Verdadero negativo 9.404 el modelo acertó	
real: paga				real: paga			
sensibilidad 0,30 · especificidad 0,996				sensibilidad 0,60 · especificidad 0,973			

El modelo es el mismo: solo cambia desde qué probabilidad se dice "no paga". Con 0,2 se detectan 199 morosos en vez de 100, y se molesta a 263 buenos clientes en vez de 42.

Sensibilidad y especificidad según el umbral

Bajar el umbral sube la sensibilidad y baja la especificidad: no se pueden subir las dos a la vez



umbral	VP	FN	FP	sensib.	especif.
0,5	100	233	42	0,30	0,996
0,3	167	166	147	0,50	0,985
0,2	199	134	263	0,60	0,973
0,1	244	89	550	0,73	0,943

Cada umbral es un balance entre los dos errores. ¿Cuál elegir? Depende de cuánto cuesta cada error, y eso no lo dice el modelo: lo dice el problema.

El costo de cada error

Con esos costos, el umbral 0,5 no es el mejor: el costo total es mínimo con un umbral mucho más bajo

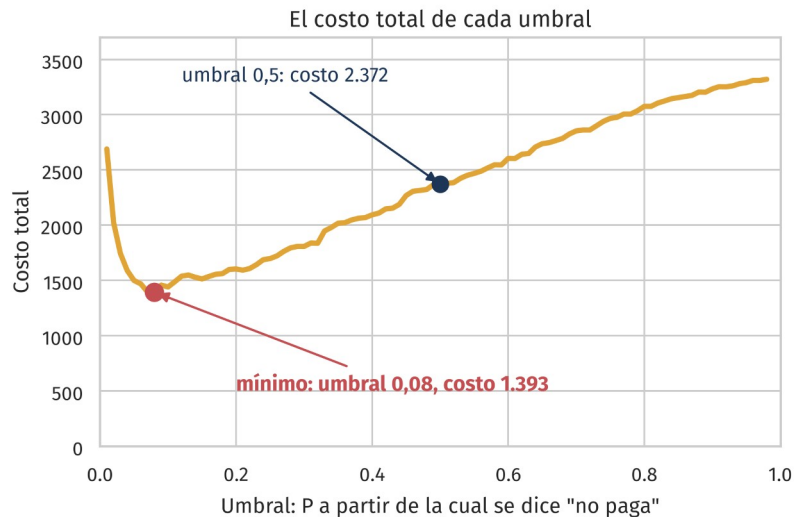
Supuesto: los dos errores no cuestan lo mismo

falso negativo (darle la tarjeta a quien no paga): cuesta 10

falso positivo (negársela a quien sí paga): cuesta 1

costo total = $10 \cdot FN + 1 \cdot FP$

umbral	$10 \cdot FN + FP$	costo total
0,50	$10 \cdot 233 + 42$	2.372
0,30	$10 \cdot 166 + 147$	1.807
0,20	$10 \cdot 134 + 263$	1.603
0,10	$10 \cdot 89 + 550$	1.440
0,08	$10 \cdot 72 + 673$	1.393



Si dejar pasar a un moroso cuesta 10 veces lo que cuesta rechazar a un buen cliente, el costo total baja de 2.372 con umbral 0,5 a 1.393 con umbral 0,08. Los costos son un supuesto: en un proyecto real los define el problema.

Cliente por cliente: ¿aprobar o rechazar?

La decisión de menor costo, cliente por cliente: se aprueba si aprobar cuesta menos que rechazar

P es la probabilidad de no pagar que el modelo le da a un cliente. Para ese cliente hay dos decisiones:

aprobar (decir "paga"): si el cliente no paga, lo que pasa con probabilidad P, se pierden 10. Costo esperado: $P \cdot 10$

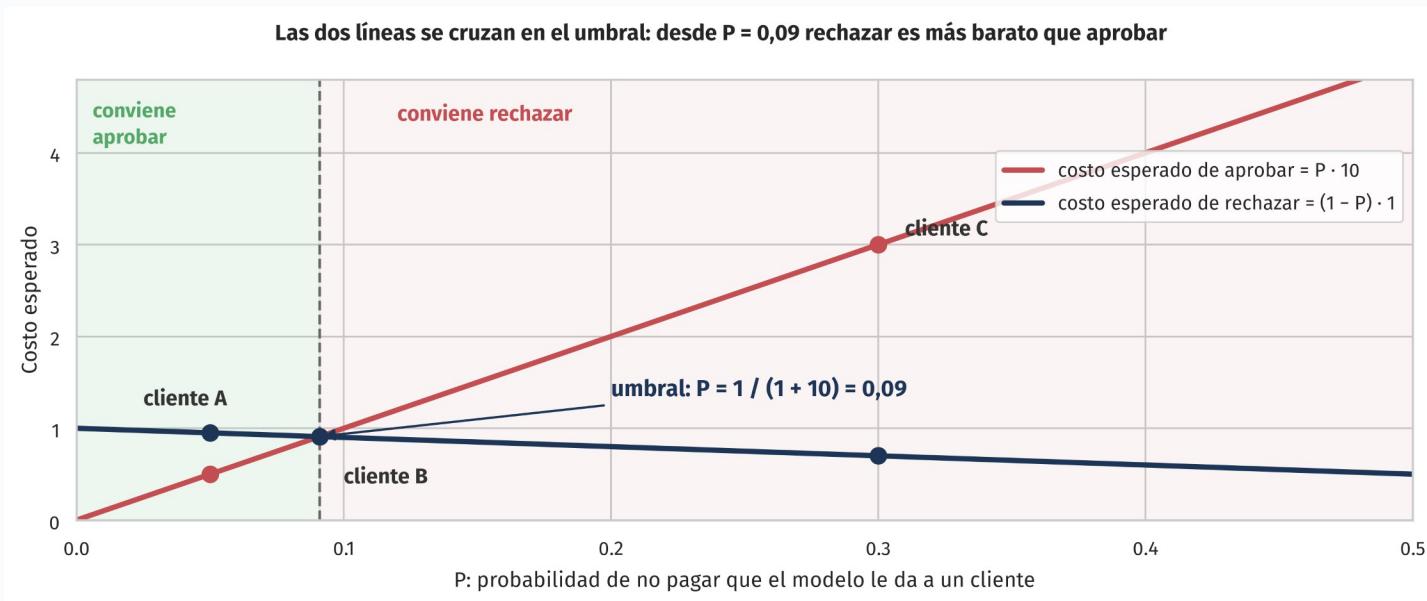
rechazar (decir "no paga"): si iba a pagar, lo que pasa con probabilidad $1 - P$, se pierde 1. Costo esperado: $(1 - P) \cdot 1$

cliente	P del modelo	aprobar: $P \cdot 10$	rechazar: $(1 - P) \cdot 1$	conviene
A	0,05	0,50	0,95	aprobar
B	0,09	0,91	0,91	da lo mismo
C	0,30	3,00	0,70	rechazar

P no es el umbral: es la probabilidad que el modelo le da a cada cliente. Con esa P se calcula cuánto cuesta en promedio cada decisión, y se elige la más barata. Al cliente A conviene aprobarlo; al C, rechazarlo; con el B da lo mismo.

El umbral de menor costo

El umbral según los costos
 $FP / (FP + FN)$



Las mismas cuentas para todos los P posibles. El umbral es el P del cliente B, donde aprobar y rechazar cuestan lo mismo: $1 / (1 + 10) = 0,09$. Con costos iguales, las líneas se cruzarían en $1 / (1 + 1) = 0,5$: por eso 0,5 es el umbral por defecto.

De dónde sale la regla del umbral

De dónde sale la regla del umbral: igualar los dos costos y despejar P

El umbral es la P en que aprobar y rechazar cuestan lo mismo. Se igualan los dos costos y se despeja P:

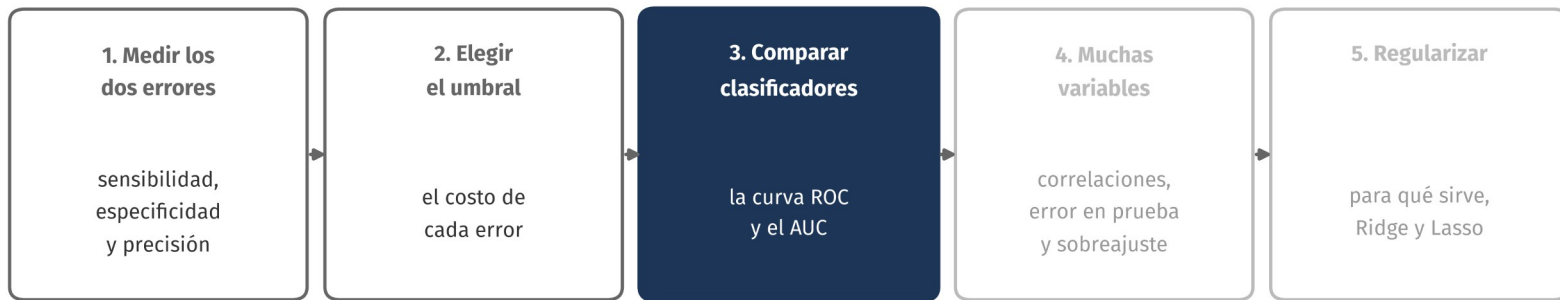
	en general	con costo FN = 10 y costo FP = 1
1. se igualan los dos costos	$(1 - P) \cdot \text{costo FP} = P \cdot \text{costo FN}$	$(1 - P) \cdot 1 = P \cdot 10$
2. se multiplica el paréntesis	$\text{costo FP} - P \cdot \text{costo FP} = P \cdot \text{costo FN}$	$1 - P = 10P$
3. los términos con P, a la derecha	$\text{costo FP} = P \cdot \text{costo FN} + P \cdot \text{costo FP}$	$1 = 10P + P$
4. P de factor común	$\text{costo FP} = P \cdot (\text{costo FN} + \text{costo FP})$	$1 = 11P$
5. se despeja P	$P = \frac{\text{costo FP}}{\text{costo FP} + \text{costo FN}}$	$P = \frac{1}{11} = 0,09$

Sobre esa P, aprobar cuesta más que rechazar: se rechaza. Bajo esa P, se aprueba. Con costos iguales, $P = 1 / 2 = 0,5$.

Cinco pasos de álgebra: igualar el costo de aprobar con el de rechazar y despejar P. Con 10 y 1 da $1 / 11 = 0,09$; con costos iguales, 0,5.

El mapa de la clase

Un modelo se juzga por sus errores donde importan: con el costo correcto y en datos que no vio



clasificar: los clientes de la tarjeta

regresión: los salarios de béisbol

¿Cuál de los dos modelos es mejor?

Con un solo umbral no se puede decir qué modelo es mejor: cada uno gana en una medida

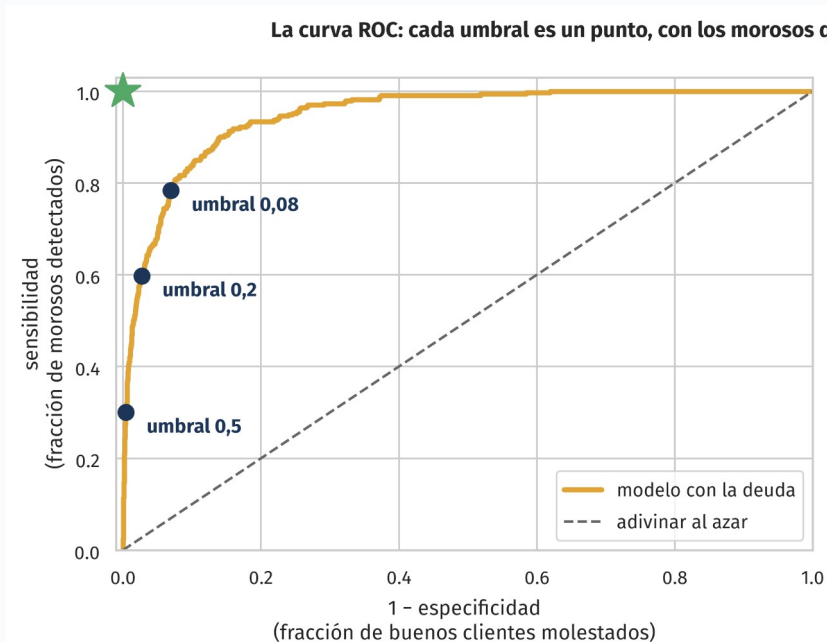
con umbral 0,5	sensibilidad (morosos detectados)	especificidad (buenos clientes en paz)
modelo con la deuda	0,30	0,996
modelo solo con estudiante	0	1

Cada modelo gana en una medida (en verde): el de la deuda detecta más morosos, y el de estudiante no molesta a ningún buen cliente, porque no detecta a nadie.

Con otro umbral los números cambian: hay que mirar todos los umbrales.

Dos modelos para predecir quién no paga: uno usa la deuda; el otro, solo si el cliente es estudiante. El umbral se elige con los costos, pero antes hay que elegir el modelo, y a veces los costos todavía no se conocen.

La curva ROC: todos los umbrales a la vez



Cada umbral es un punto de la curva:

umbral	sensibilidad (detectados)	1 - especificidad (molestados)
0,5	0,30	0,004
0,2	0,60	0,027
0,08	0,78	0,070

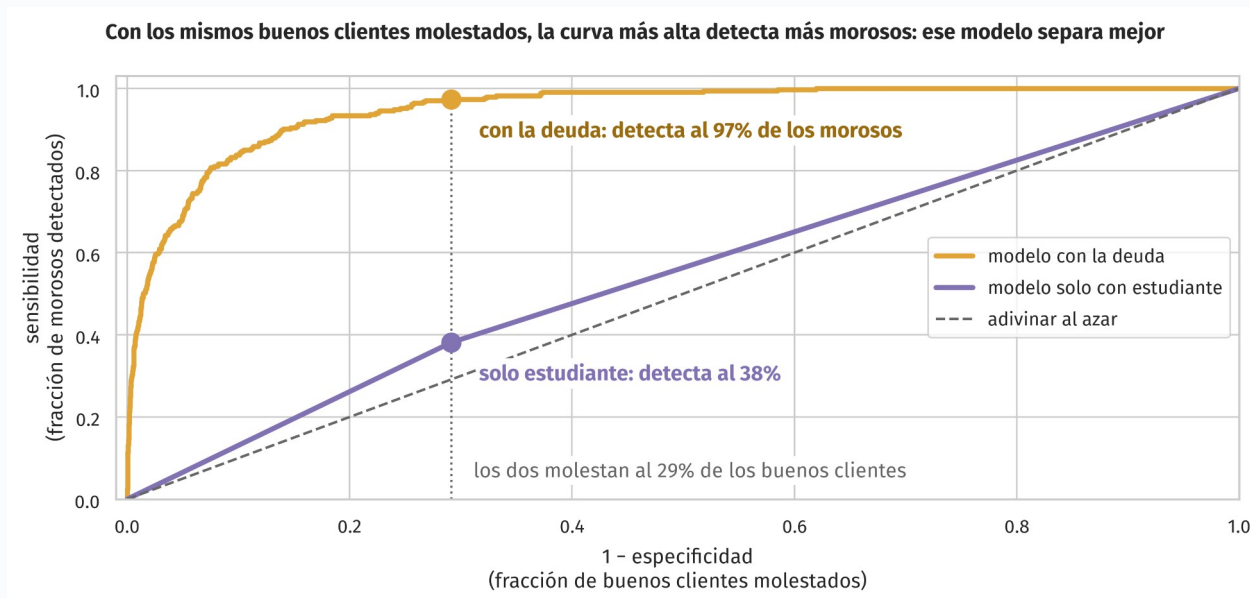
Bajar el umbral sube por la curva: más morosos detectados, pero más buenos clientes molestados.

La estrella verde es lo ideal: detectar a todos sin molestar a nadie. La diagonal es adivinar.

Mientras más se acerca la curva a la estrella, mejor separa el modelo.

La curva muestra lo que el modelo puede ofrecer con cualquier umbral: cuántos morosos detecta a cambio de cuántos buenos clientes molesta. Sirve para juzgar el modelo antes de elegir el umbral.

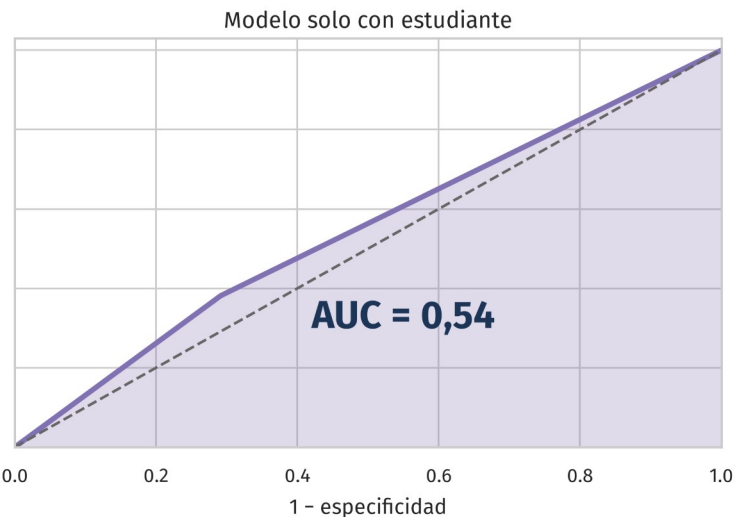
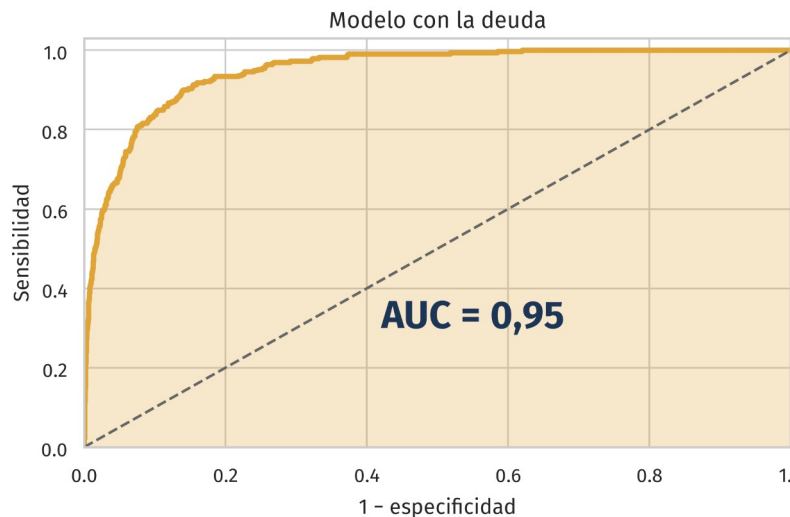
Dos modelos, dos curvas



El modelo de estudiante solo distingue estudiantes de no estudiantes, por eso su curva tiene un solo quiebre: rechazar a todos los estudiantes. La curva de la deuda va siempre por encima: ese modelo es mejor con cualquier umbral, sin importar los costos.

El AUC: la comparación en un número

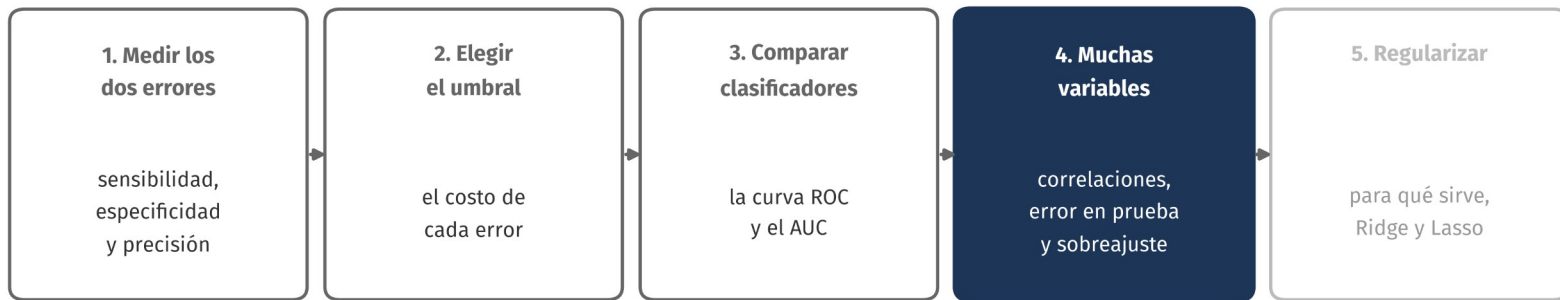
El área bajo la curva ROC (AUC) resume todos los umbrales en un número: 1 separa perfecto, 0,5 es adivinar



Mientras más alta la curva, mayor el área. El AUC se lee así: si se toma al azar un cliente que no paga y uno que paga, el modelo de la deuda le da más probabilidad al que no paga en el 95% de los pares; el de estudiante, en el 54%, casi como adivinar.

El mapa de la clase

Un modelo se juzga por sus errores donde importan: con el costo correcto y en datos que no vio



clasificar: los clientes de la tarjeta

regresión: los salarios de béisbol

Otro problema: el salario de jugadores de béisbol

Ejemplo: el salario de jugadores de béisbol de las grandes ligas, en miles de dólares de 1987 (James et al., laboratorio 6.6)

jugador	salario	temporadas	turnos	hits	jonrones	turnos_carrera	hits_carrera	division_oeste
1	475,0	14	315	81	7	3.449	835	1
2	480,0	3	479	130	18	1.624	457	1
3	500,0	11	496	141	20	5.628	1.575	0
4	91,5	2	321	87	10	396	101	0
5	750,0	11	594	169	4	4.408	1.133	1

263 jugadores y 19 variables (la tabla muestra 7):

de la temporada 1986: turnos al bate, hits, jonrones, carreras anotadas e impulsadas, bases por bolas, outs en defensa, asistencias, errores

de toda la carrera: temporadas y los mismos totales acumulados · categóricas: liga, división y liga en 1987

263 jugadores de las grandes ligas de Estados Unidos, con su salario de 1987 y 19 variables de la temporada 1986 y de su carrera (James et al., laboratorio 6.6). La respuesta es un número: volvemos a la regresión lineal.

Las variables de bateo de la temporada 1986

Las seis variables de bateo de la temporada 1986 (con los valores del jugador 1 de la tabla)

variable	qué cuenta, en la temporada 1986	jugador 1
turnos	veces que fue a batear	315
hits	veces que golpeó la pelota y llegó a una base	81
jonrones	batazos que sacaron la pelota del campo: recorre las cuatro bases y anota	7
anotadas	carreras que anotó: veces que completó la vuelta y llegó a home	24
impulsadas	carreras que anotaron sus compañeros gracias a un batazo suyo	38
bases_bolas	veces que avanzó a primera base sin batear, por cuatro lanzamientos fallados	39

Seis cuentas de lo que hizo el jugador al batear durante 1986.

Las mismas cuentas, en toda la carrera

Las mismas seis cuentas, sumadas en toda la carrera del jugador

temporada 1986	jugador 1	toda la carrera	jugador 1
turnos	315	turnos_carrera	3.449
hits	81	hits_carrera	835
jonrones	7	jonrones_carrera	69
anotadas	24	anotadas_carrera	321
impulsadas	38	impulsadas_carrera	414
bases_bolas	39	bases_bolas_carrera	375

temporadas: años en las grandes ligas (el jugador 1 lleva 14). Con más temporadas, más de todo en la carrera.

Siete variables más: las temporadas en las grandes ligas y los seis totales acumulados.

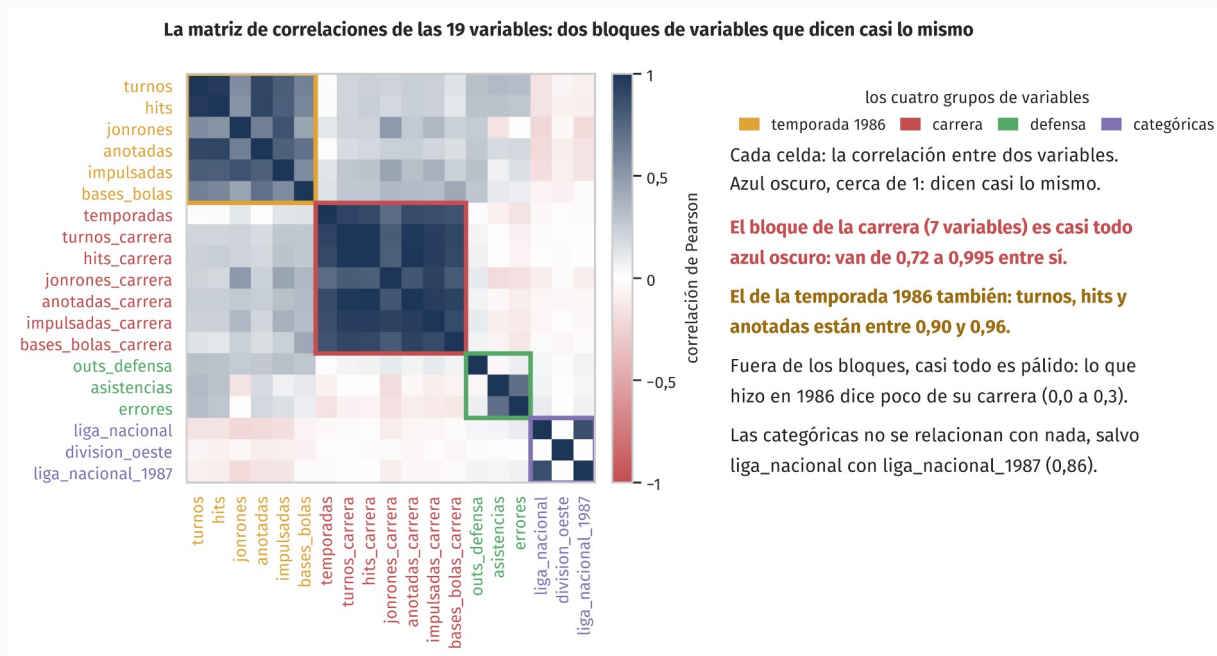
Defensa, categóricas y la respuesta

Las tres variables de defensa, las tres categóricas y la respuesta

variable	qué cuenta, en la temporada 1986	jugador 1
outs_defensa	rivales que eliminó cuando su equipo defendía (atrapó la pelota o llegó antes a la base)	632
asistencias	jugadas en que ayudó a eliminar a un rival (por ejemplo, le lanzó la pelota a quien hizo el out)	43
errores	jugadas en defensa que falló y dejaron avanzar al rival	10
liga_nacional	1 si en 1986 jugó en la Liga Nacional, 0 si en la Americana (las dos grandes ligas)	1
division_oeste	1 si su equipo era de la división Oeste de su liga, 0 si de la Este	1
liga_nacional_1987	la misma variable de liga, pero al empezar 1987	1
salario	lo que queremos predecir: el salario anual de 1987, en miles de dólares	475,0

Con estas se completan las 19 variables. El salario es lo que el modelo debe predecir.

¿Qué variables dicen lo mismo?



Dos bloques oscuros: las seis de la temporada 1986 entre sí, y las siete de la carrera entre sí. Entre bloques, casi nada.

El bloque de la carrera, con números

Acercamiento al bloque de la carrera: correlaciones entre 0,72 y 0,995

temporadas	1,00	0,92	0,90	0,72	0,88	0,86	0,84
turnos_carrera	0,92	1,00	0,995	0,80	0,98	0,95	0,91
hits_carrera	0,90	0,995	1,00	0,79	0,98	0,95	0,89
jonrones_carrera	0,72	0,80	0,79	1,00	0,83	0,93	0,81
anotadas_carrera	0,88	0,98	0,98	0,83	1,00	0,95	0,93
impulsadas_carrera	0,86	0,95	0,95	0,93	0,95	1,00	0,89
bases_bolas_carrera	0,84	0,91	0,89	0,81	0,93	0,89	1,00

temporadas
turnos_carrera
hits_carrera
jonrones_carrera
anotadas_carrera
impulsadas_carrera
bases_bolas_carrera

Las siete variables de la carrera miden, en el fondo, lo mismo: cuánto ha jugado.

Con más temporadas se acumulan más turnos, más hits, más carreras.

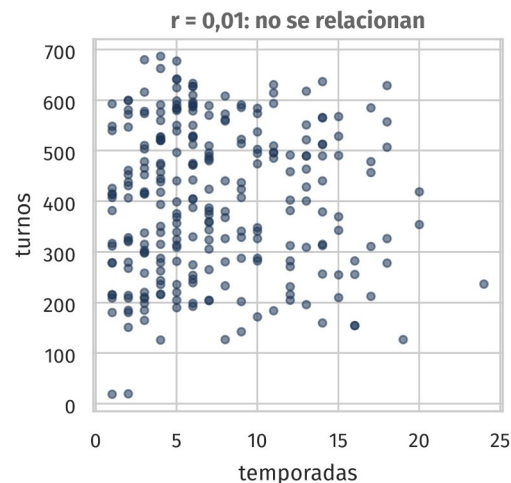
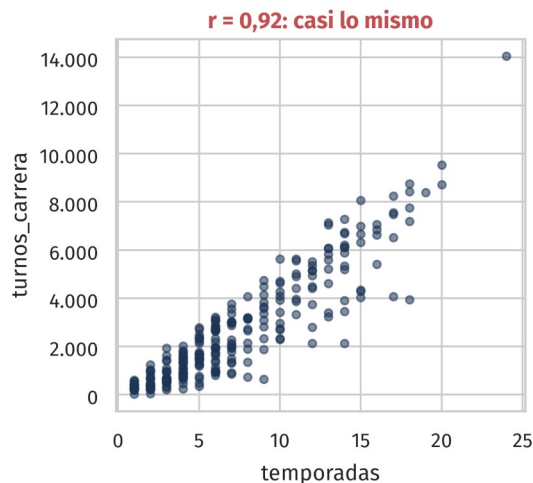
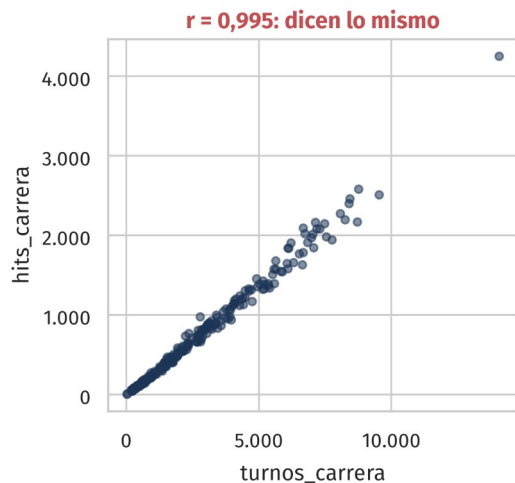
Turnos y hits de la carrera: correlación 0,995.
Saber una es saber la otra.

Para el modelo esto es un problema: si dos variables dicen lo mismo, no hay cómo saber a cuál darle el mérito.

Siete variables que miden lo mismo: cuánto ha jugado. Turnos y hits de la carrera: 0,995.

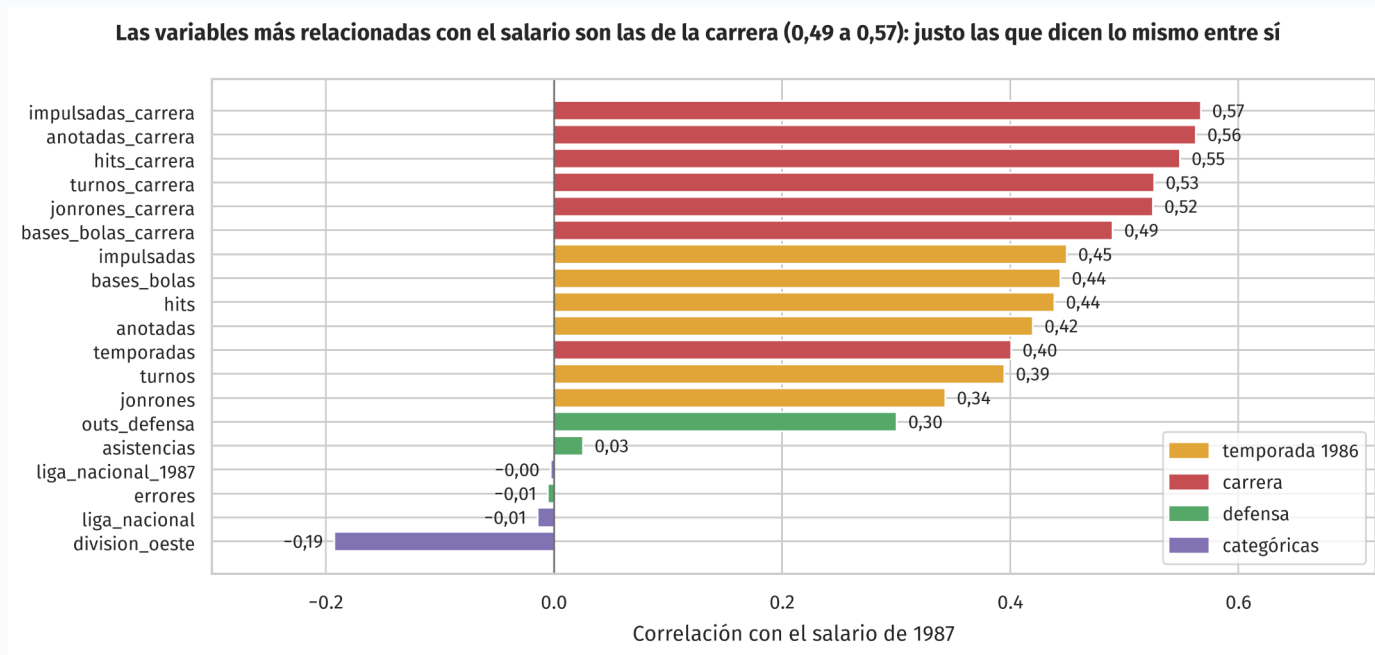
Tres pares de variables, jugador por jugador

Tres pares de variables, un punto por jugador: qué significa una correlación de 0,995, de 0,92 y de 0,01



Con correlación 0,995, saber una variable es saber la otra: el modelo no tiene cómo repartir el mérito entre las dos.

¿Y con el salario?



Ninguna variable sola explica el salario. Las más relacionadas son las de la carrera: justo las que dicen lo mismo entre sí.

Entrenamiento y prueba: por qué se parte

Antes de medir errores: una mitad para ajustar y otra para medir

263 jugadores con salario, en el orden del archivo

263

se barajan al azar y se parten en dos mitades

antes de ajustar nada:

131 de entrenamiento

132 de prueba

para ajustar el modelo
(y todo lo que se aprende:
estandarizar, elegir λ)

para medir el error,
una sola vez, al final

El modelo se ajusta a los jugadores que ve, con el azar de esa muestra incluido.

Si el error se midiera con esos mismos jugadores, saldría optimista: el modelo ya los vio.

Por eso se aparta una mitad antes de ajustar: la prueba. El error que vale es el que se mide ahí.

La prueba no se toca hasta el final: ni para ajustar, ni para estandarizar, ni para elegir λ .

Es lo mismo que se hizo en la clase 6 con los clientes de la tarjeta.

Se barajan los 263 y se parten en dos mitades antes de ajustar nada: 131 para ajustar y 132 para medir. La prueba no se toca hasta el final.

Cómo se parte en el código

Cómo se parte en el código: `train_test_split`, de scikit-learn

Una sola línea. Lo que sale, a la izquierda del `=`; lo que entra, entre paréntesis.

```
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.5, random_state=1)
```

Salen: `X_ent, y_ent`

variables (`X_ent`) y salario (`y_ent`)
de los 131 de entrenamiento:
con estos se ajusta todo

Salen: `X_pru, y_pru`

variables (`X_pru`) y salario (`y_pru`)
de los 132 de prueba:
con estos solo se mide el error

Entra: `X, y`

`X`: la tabla de 263 jugadores
por 19 variables
`y`: el salario de los 263

Entran: dos opciones

`test_size=0.5`: la mitad a prueba
`random_state=1`: fija el sorteo,
el reparto es siempre el mismo

Regla: todo lo que se aprende (el modelo, la media y la desviación para estandarizar, el λ) se aprende con las piezas `_ent`.

Con las piezas `_pru` solo se mide el error, una vez, al final.

Entra la tabla `X` y el salario `y`; salen cuatro piezas, dos de entrenamiento y dos de prueba. En el notebook, esta línea abre la sección 8.

El error de una regresión: el RMSE

El RMSE: la raíz del promedio, sobre los jugadores, de los residuos al cuadrado

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

y_i

el salario real del jugador i

$\hat{y}_i$

el salario que el modelo predice para el jugador i

$y_i - \hat{y}_i$

el residuo: cuánto se equivocó el modelo con ese jugador

$(y_i - \hat{y}_i)^2$

el residuo al cuadrado: así los errores hacia arriba y hacia abajo no se cancelan

$\frac{1}{n} \sum_{i=1}^n$

el promedio sobre los n jugadores: se suman los n cuadrados y se divide por n

$\sqrt{\dots}$

la raíz, para volver a miles de dólares

El RMSE es el tamaño típico del error del modelo, en miles de dólares, como el salario.

Como en la clase 6: 131 jugadores para ajustar y 132 de prueba, y el error se mide con los de prueba, que el modelo no vio.

El RMSE paso a paso

El RMSE paso a paso, con cinco jugadores de prueba y el modelo de mínimos cuadrados (salarios en miles de dólares)

jugador	salario real y	predicho $\hat{y}$	residuo $y - \hat{y}$	residuo ² $(y - \hat{y})^2$
1	500	620	-120	14.400
2	700	315	385	148.225
3	700	753	-53	2.809
4	210	492	-282	79.524
5	105	341	-236	55.696
			suma	300.654

1. Promedio de los residuos al cuadrado:

$$300.654 / 5 = 60.131$$

la suma de la tabla, dividida por los 5 jugadores

2. Raíz, para volver a miles de dólares:

$$\text{RMSE} = \sqrt{60.131} = 245$$

Con los 132 jugadores de prueba: RMSE = 342

Sin modelo, prediciendo siempre el salario medio: 416

El modelo se equivoca típicamente en 342 mil dólares.

Con los 132 jugadores de prueba, el error típico del modelo es de 342 mil dólares; sin modelo, de 416 mil.

Estandarizar las variables

Estandarizar: a cada variable se le resta su media y se divide por su desviación estándar

jugador	temporadas	turnos carrera	temporadas (z)	turnos carrera (z)
1	17	4.061	2,01	0,62
2	11	5.562	0,75	1,27
3	5	2.364	-0,50	-0,11
media	7,4	2.625	0	0
desv. estándar	4,8	2.313	1	1

$$z = \frac{x - \text{media}}{\text{desviación estándar}}$$

Sin estandarizar, una temporada y un turno al bate no son comparables, y tampoco sus coeficientes: uno es por temporada y el otro por turno.

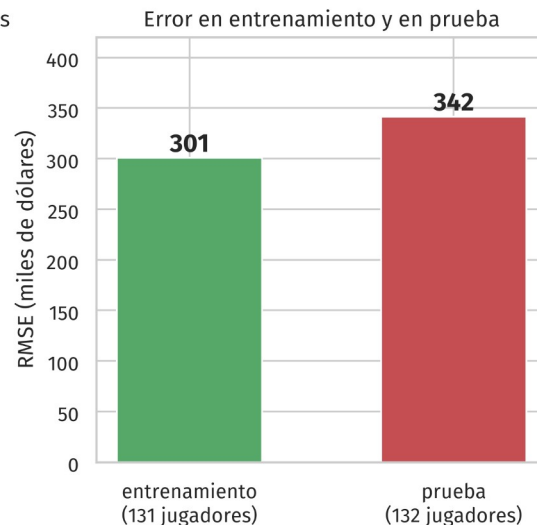
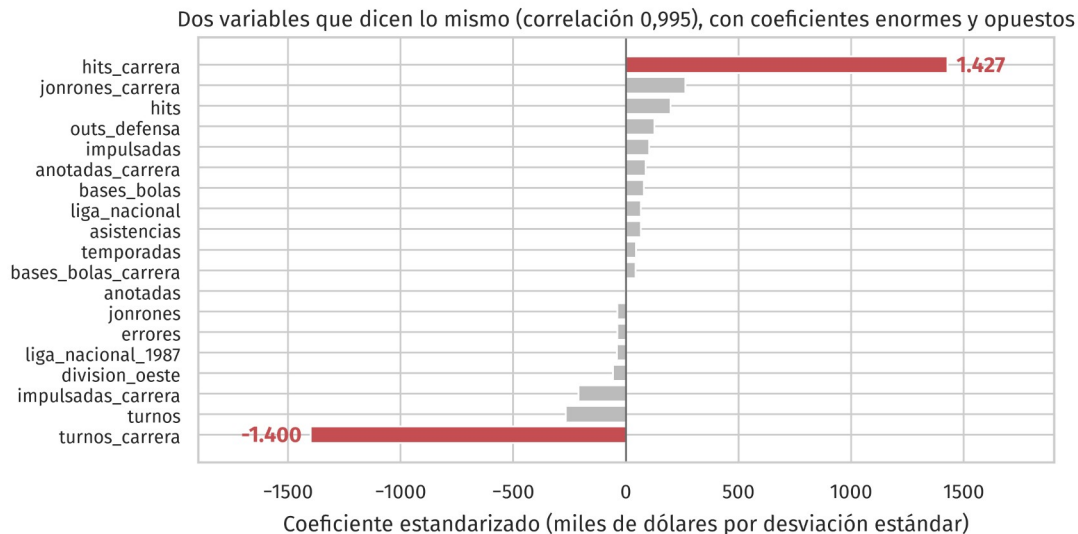
Después, todas tienen media 0 y desviación 1, y cada coeficiente dice cuánto cambia el salario por una desviación estándar de la variable.

La media y la desviación se calculan con los datos de entrenamiento, y se aplican también a los de prueba.

Estandarizadas, cada coeficiente dice cuánto cambia el salario por una desviación estándar, y se pueden comparar. La media y la desviación salen del entrenamiento.

Mínimos cuadrados con 19 variables

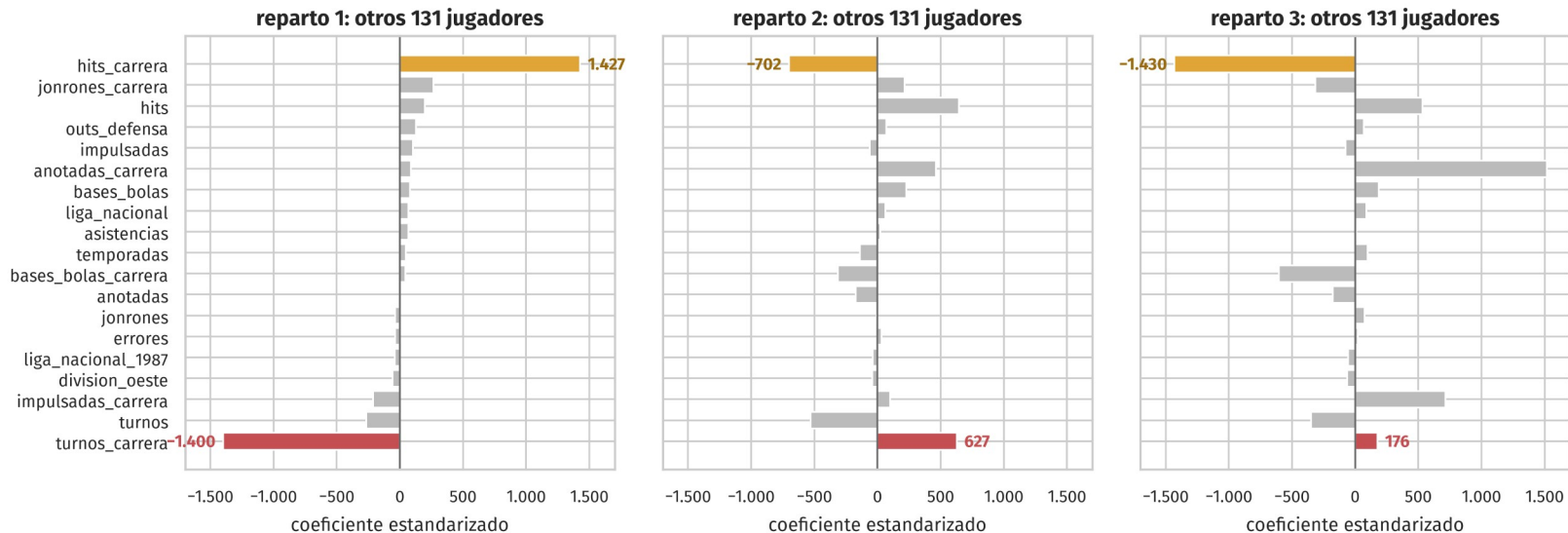
Mínimos cuadrados con las 19 variables: se ajusta al azar de sus 131 jugadores



A turnos y hits de la carrera, con correlación 0,995, les da coeficientes enormes de signo opuesto que se cancelan. Se equivoca en 301 con los jugadores que vio y en 342 con los nuevos.

Otros jugadores, otros coeficientes

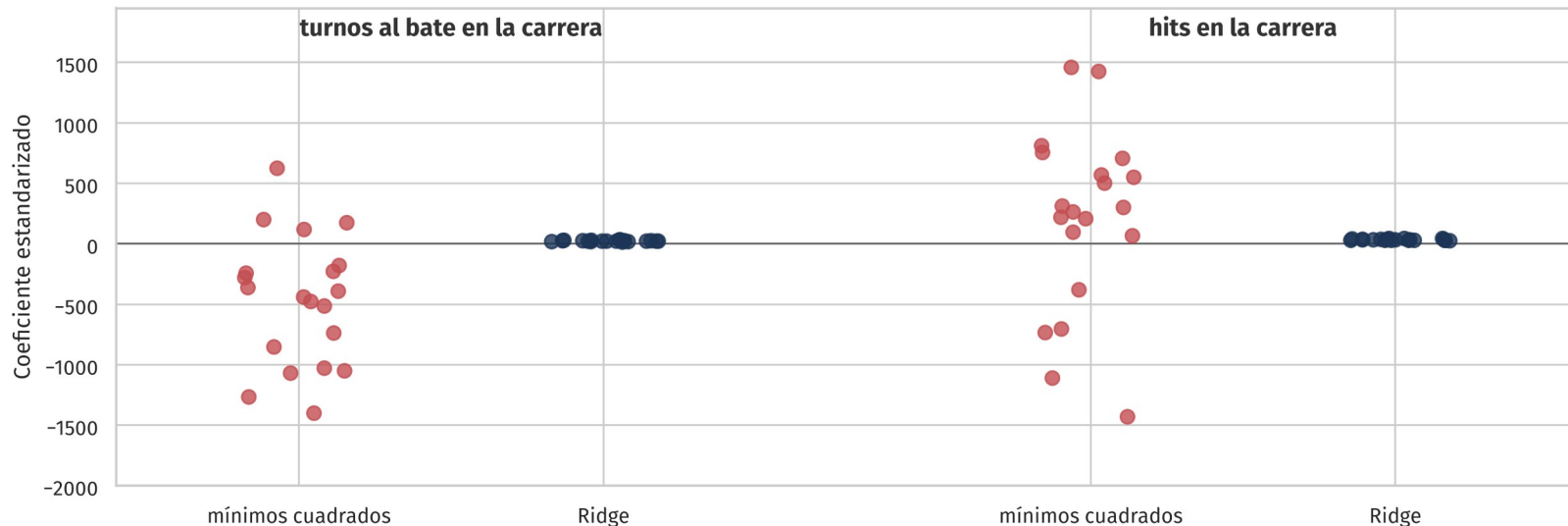
La misma regresión de mínimos cuadrados con tres repartos distintos: turnos y hits de la carrera cambian hasta de signo



Tres repartos distintos de los 263 jugadores y la misma regresión: turnos y hits de la carrera cambian de tamaño y hasta de signo según qué 131 jugadores tocaron.

Los coeficientes varían mucho entre muestras

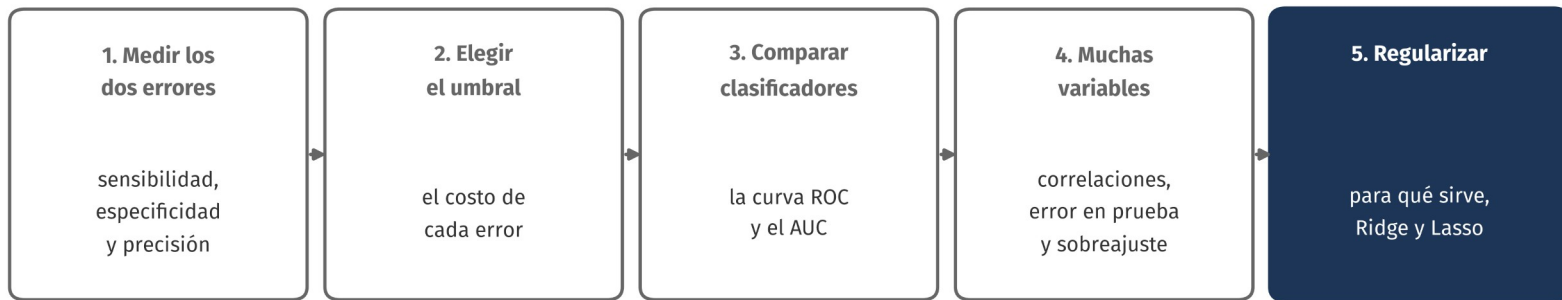
Un punto por reparto: con 20 grupos distintos de 131 jugadores, mínimos cuadrados cambia hasta de signo; Ridge casi no se mueve



Cada punto es un reparto distinto. Con mínimos cuadrados, el coeficiente de turnos de la carrera va de -1.400 a 627 según qué 131 jugadores tocaron: ese es el problema. Con Ridge, de 15 a 36.

El mapa de la clase

Un modelo se juzga por sus errores donde importan: con el costo correcto y en datos que no vio



clasificar: los clientes de la tarjeta

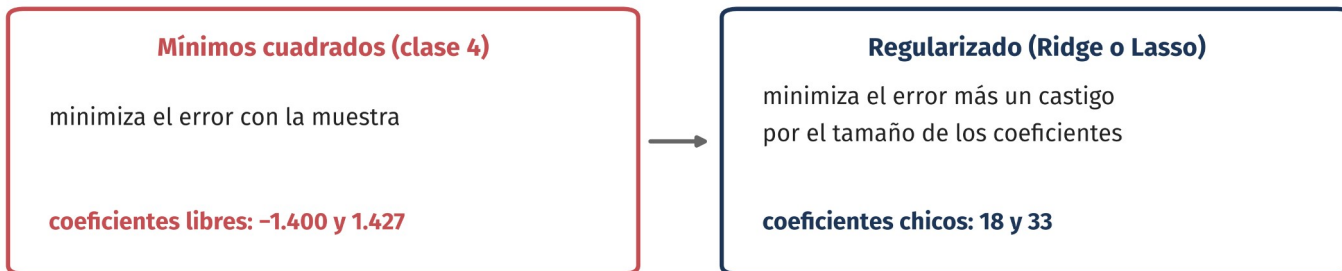
regresión: los salarios de béisbol

¿Qué es regularizar?

Qué es regularizar

**Regularizar un modelo es ajustarlo con todas las variables,
pero impidiendo que sus coeficientes crezcan: se los empuja hacia 0.**

James et al. (2013, p. 214): "constrains or regularizes the coefficient estimates, or equivalently, ... shrinks the coefficient estimates towards zero".



Pierde un poco con los datos que vio y gana con los nuevos: coeficientes y predicciones que cambian menos entre muestras.

Los coeficientes ya no toman el valor que mejor ajusta la muestra: se les impide crecer. Cuánto se les impide lo fija λ , un número que se elige con los datos.

¿Para qué regularizar?

Regularizar es obligar al modelo a usar coeficientes chicos. Lo que cambia con Ridge:

en miles de dólares; en verde, el mejor de cada fila	mínimos cuadrados	Ridge
coeficientes de turnos y hits de la carrera	-1.400 y 1.427	18 y 33
cuánto cambia la predicción de un jugador si se usa otra muestra	± 97	± 48
error con los 131 jugadores que el modelo vio	301	340
error con los 132 jugadores nuevos	342	316

Coeficientes chicos: el modelo depende menos de su muestra y se equivoca menos con lo nuevo.

Con coeficientes chicos, el modelo depende menos de su muestra: pierde un poco con los jugadores que vio y gana con los nuevos.

¿Cuándo conviene regularizar?

Conviene regularizar

- cuando el objetivo es predecir casos nuevos
- cuando hay muchas variables para pocos datos (aquí, 19 variables y 131 jugadores)
- cuando hay variables que dicen casi lo mismo (turnos y hits de la carrera: 0,995)

No hace falta, o no sirve

- con pocas variables y muchos datos: el resultado es casi el de mínimos cuadrados (la clase 6: 2 variables, 10.000 clientes)
- para leer coeficientes con valores p, como en la clase 5: aquí se achican a propósito

¿Cómo me doy cuenta de que hace falta?

Cómo darse cuenta de que hace falta regularizar: cuatro señales, con los números de este caso

1. Muchas variables para pocos datos

19 variables y 131 jugadores para ajustar:
cada variable se ajusta con muy pocos datos.

2. Variables que dicen lo mismo

correlaciones cerca de 1 (turnos y hits de la carrera: 0,995)
y coeficientes enormes y opuestos (-1.400 y 1.427).

3. Error en prueba mucho mayor que en entrenamiento

301 con los jugadores que vio, 342 con los nuevos:
el modelo se ajustó al azar de su muestra.

4. Coeficientes que cambian con la muestra

con otros repartos, turnos de la carrera va de -1.400 a 627
(o con bootstrap, como en la clase 5).

La regla práctica: no hay que adivinar.

Si el objetivo es predecir, se prueban Ridge y Lasso con λ elegido por validación cruzada y se compara el error en prueba. Si no hacían falta, la validación cruzada elige un λ casi 0 y devuelve mínimos cuadrados: no se pierde nada por probar.

Cuatro señales, todas presentes aquí. Y si hay duda, se prueba: con λ elegido por validación cruzada, un modelo que no necesitaba regularización vuelve a mínimos cuadrados.

Cómo se escribe: el error más un castigo

Cómo se escribe: al error del modelo se le suma un castigo por el tamaño de los coeficientes

Mínimos cuadrados (clase 4):

$$\text{minimizar } \sum_i (y_i - \hat{y}_i)^2$$

el error del modelo

Ridge:

$$\text{minimizar } \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j b_j^2$$

más el castigo

Lasso:

$$\text{minimizar } \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j |b_j|$$

i recorre los jugadores, de 1 a 131: el error se suma sobre los jugadores. j recorre las variables, de 1 a 19: el castigo se suma sobre los coeficientes.

Con dos coeficientes, 3 y -4:

Ridge castiga $3^2 + (-4)^2 = 9 + 16 = 25$

Lasso castiga $|3| + |-4| = 3 + 4 = 7$

λ decide cuánto pesa el castigo: más λ , coeficientes más chicos. Como el castigo suma coeficientes de variables distintas, se usan las variables estandarizadas.

Qué es cada símbolo

Qué es cada símbolo

$$\sum_i (y_i - \hat{y}_i)^2$$

la suma de los residuos al cuadrado: qué tan bien se ajusta el modelo a los 131 jugadores

i

el número del jugador, de 1 a 131: la suma del error recorre los jugadores

b_j

el coeficiente de la variable j , con las variables estandarizadas (el intercepto no se castiga)

j

el número de la variable, de 1 a 19: la suma del castigo recorre los coeficientes

λ (lambda)

cuánto pesa el castigo: más λ , coeficientes más chicos (scikit-learn lo llama alpha)

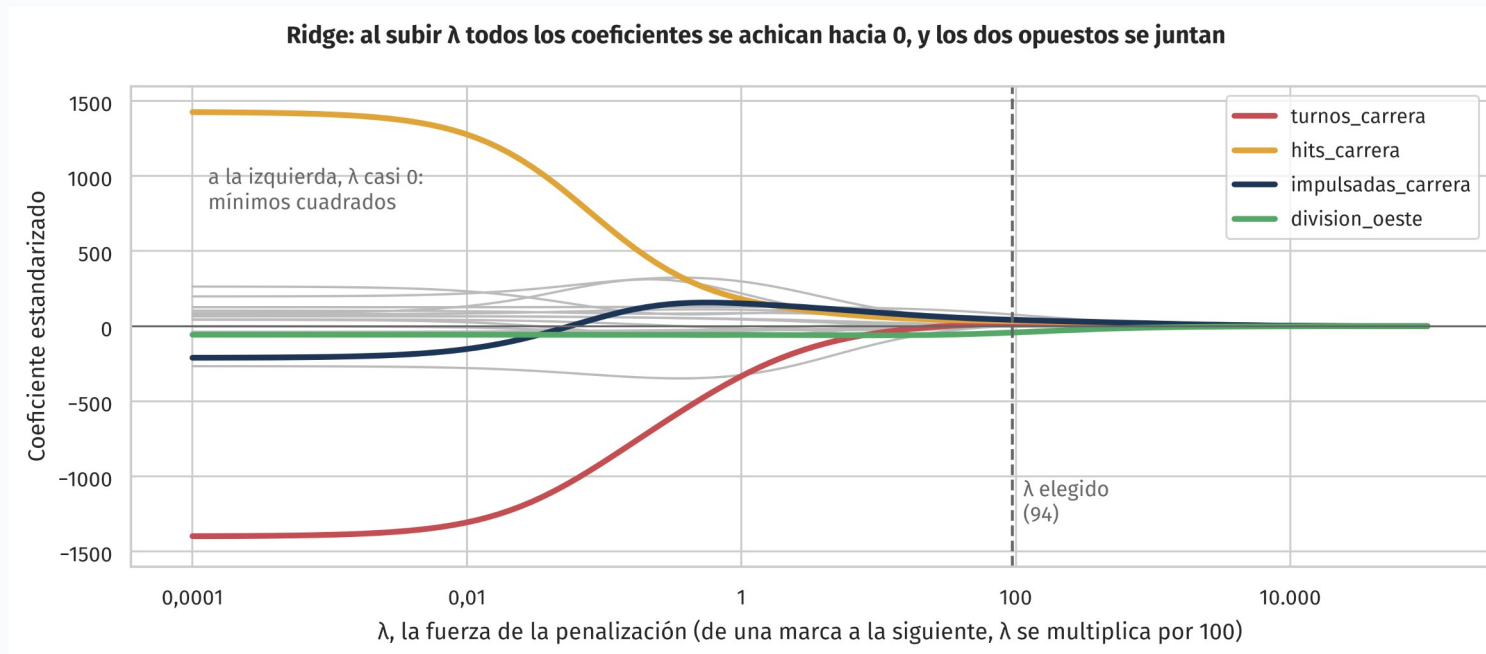
$\lambda = 0$

no hay castigo: es mínimos cuadrados

λ muy grande

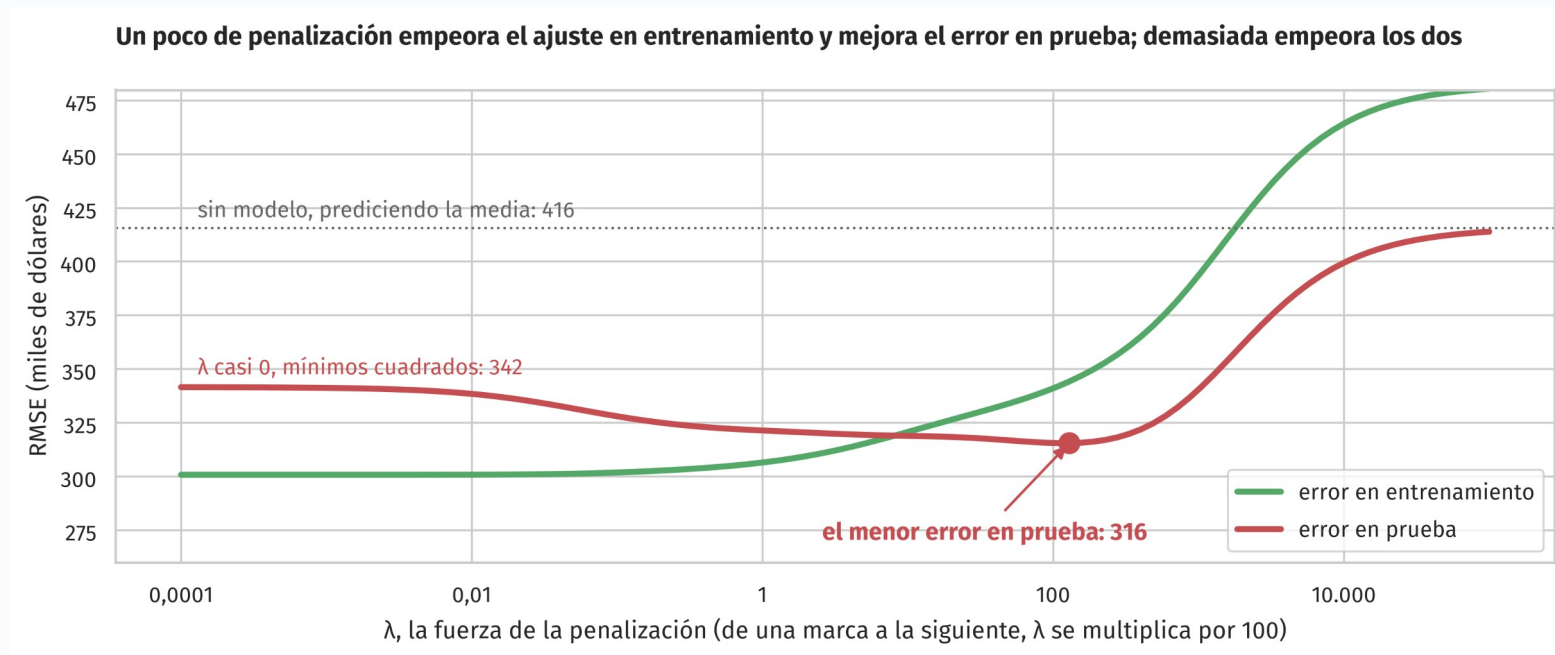
todos los coeficientes van a 0 y el modelo predice el salario promedio

Ridge: todos los coeficientes se achican



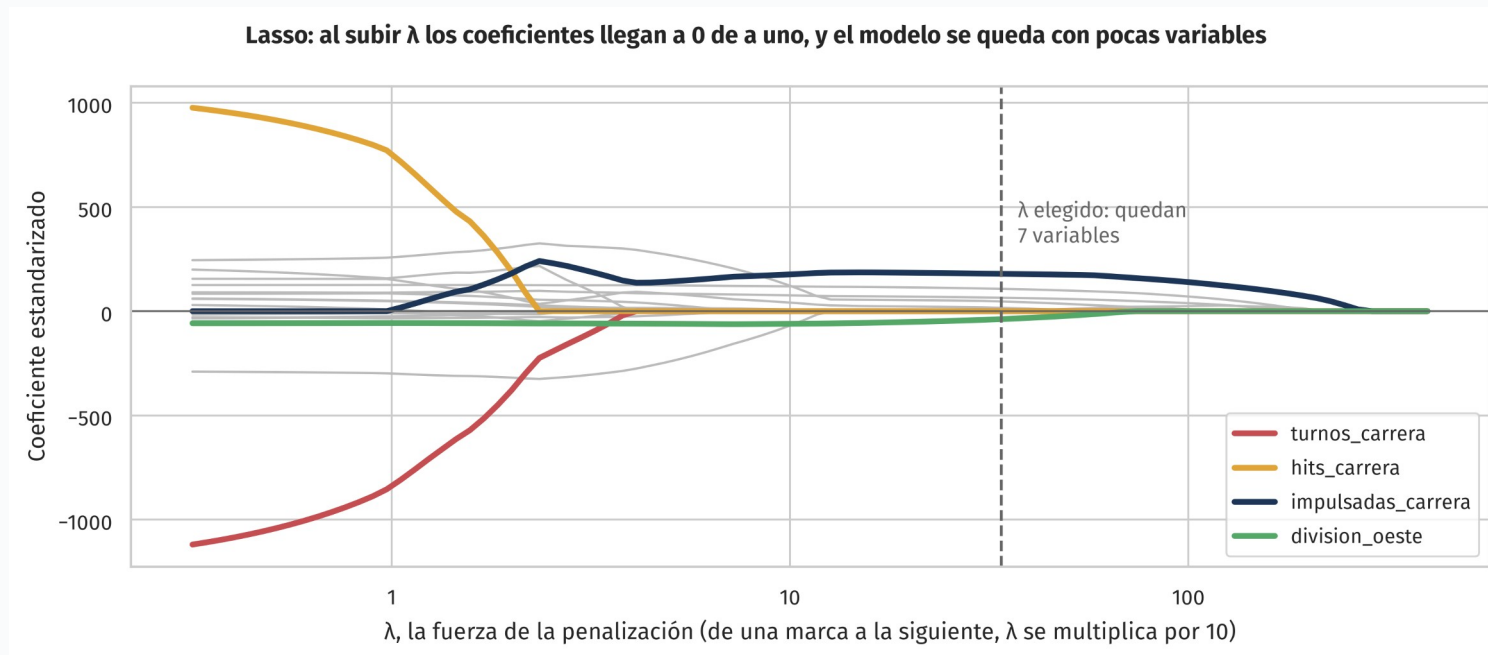
A la izquierda, λ casi 0: mínimos cuadrados, con los opuestos de -1.400 y 1.427. Al subir λ se achican y se juntan; ninguno llega exactamente a 0.

El error de Ridge según λ



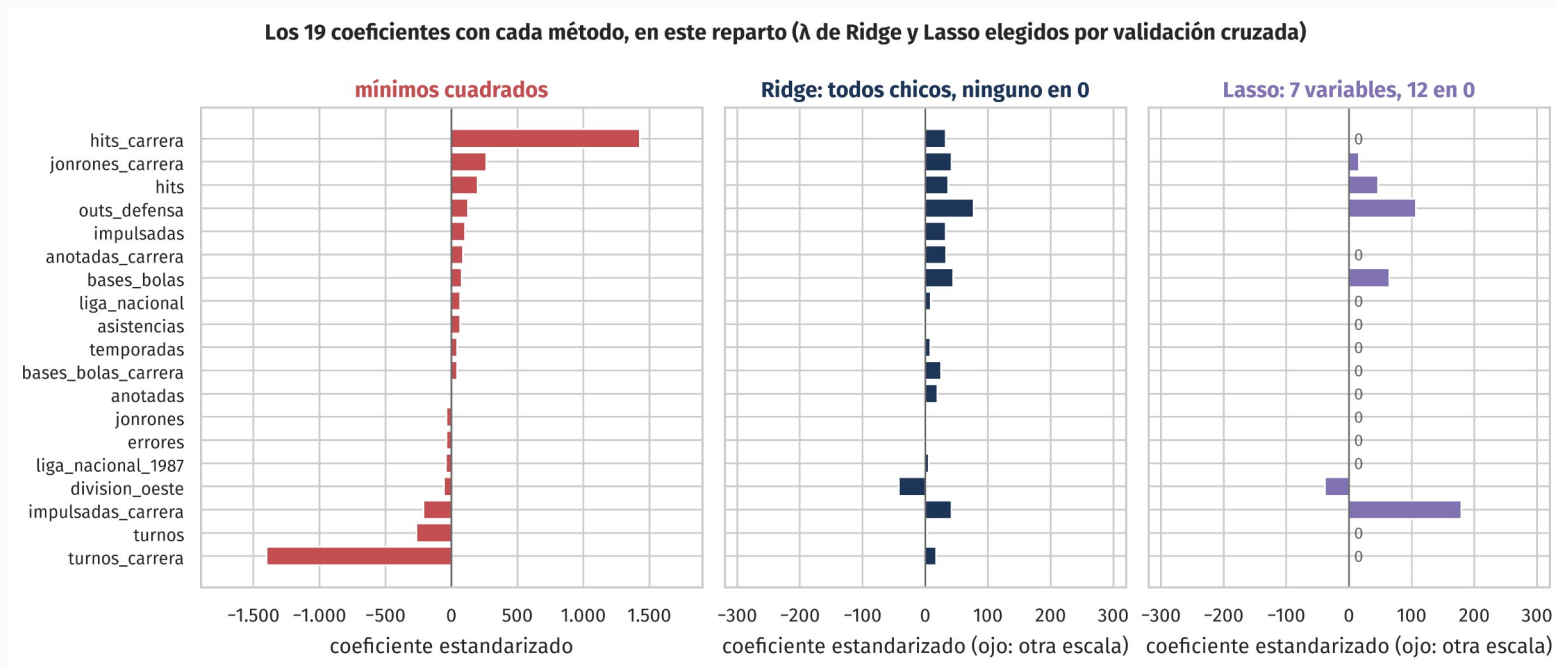
El error en entrenamiento solo sube con λ . El de prueba baja de 342 a 316 y después sube. El λ no se elige mirando la prueba, sino con validación cruzada.

Lasso: los coeficientes se apagan de a uno



Al subir λ , los coeficientes llegan exactamente a 0 de a uno: el modelo elige variables. Con el λ elegido quedan 7 de las 19, como en el libro.

Los 19 coeficientes con cada método



Ridge deja los 19 chicos y distintos de 0; Lasso deja 7 y lleva 12 exactamente a 0.

¿Qué es la validación cruzada?

- ▶ **Una forma de elegir λ sin tocar los datos de prueba.**
- ▶ El problema: la prueba es para la medición final, y el error de entrenamiento siempre prefiere $\lambda = 0$.
- ▶ La idea: apartar un grupo de jugadores, ajustar el modelo con el resto y medir el error en el grupo apartado. Se repite rotando el grupo, de modo que cada grupo se use como conjunto de validación exactamente una vez.
- ▶ Con cinco grupos: por cada λ , cinco errores y su promedio. Gana el λ de menor promedio (James et al., 2013, p. 181).

Cuántos jugadores hay en cada paso

Cuántos jugadores hay en cada paso: el ancho de cada bloque es proporcional a la cantidad

El archivo Hitters: 322 jugadores



Reparto al azar de los 263 (train_test_split, mitad y mitad)



Validación cruzada: los 131 de entrenamiento se cortan en cinco grupos (la prueba no entra)



Vuelta 1 de 5: se ajusta con los grupos 2 a 5 y se mide el error en el grupo 1



Las otras cuatro vueltas igual, rotando el grupo que valida: cada grupo valida una vez. Cinco errores por λ , y se promedian.

Con el λ elegido: se ajusta una vez más con los 131 y se mide UNA sola vez en los 132 de prueba



Los cinco grupos salen solo de los 131 de entrenamiento: 27, 26, 26, 26 y 26. En cada vuelta, cuatro grupos ajustan y uno valida. Los 132 de prueba se usan una sola vez, al final.

Cómo elegir λ : validación cruzada

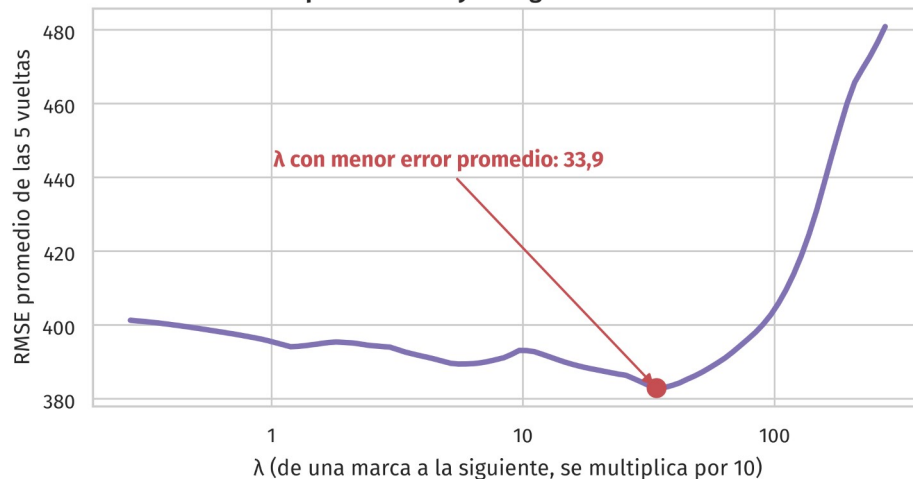
Elegir λ con validación cruzada: sin tocar los datos de prueba

Cinco partes: cada una valida una vez

vuelta 1	valida	ajusta	ajusta	ajusta	ajusta
vuelta 2	ajusta	valida	ajusta	ajusta	ajusta
vuelta 3	ajusta	ajusta	valida	ajusta	ajusta
vuelta 4	ajusta	ajusta	ajusta	valida	ajusta
vuelta 5	ajusta	ajusta	ajusta	ajusta	valida

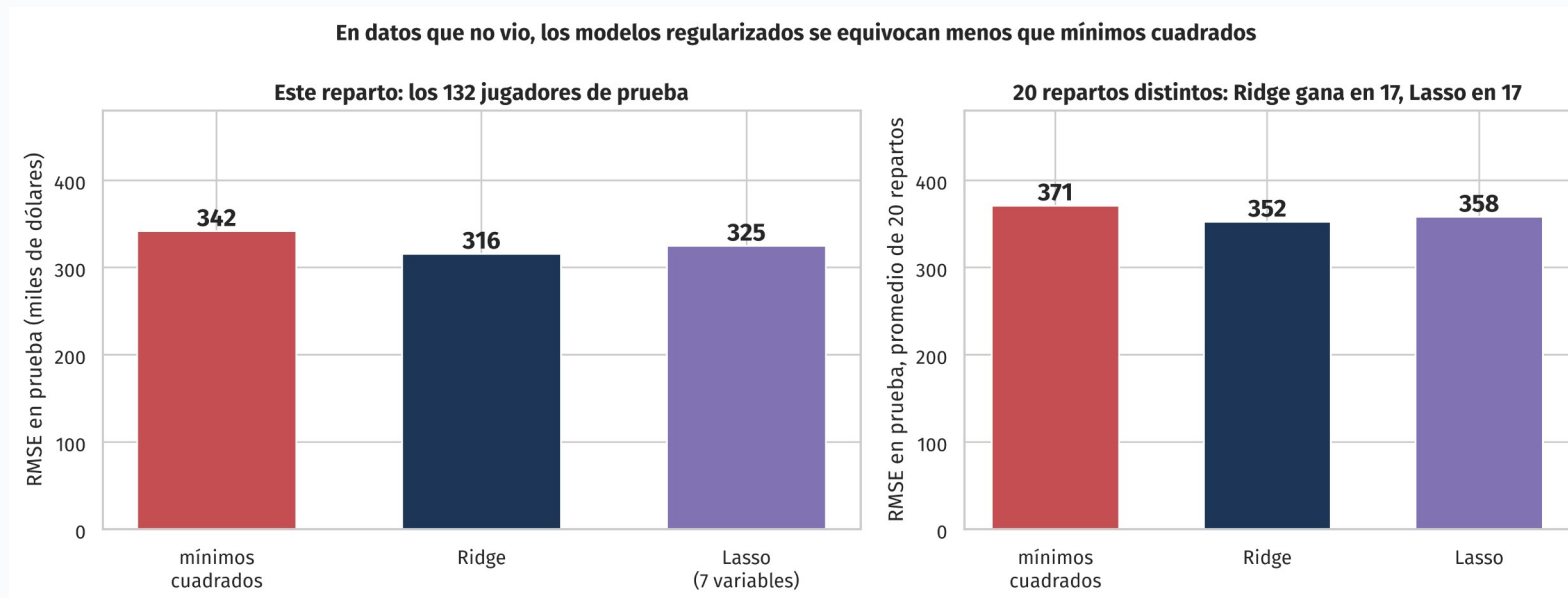
solo con los 131 jugadores de entrenamiento

Se prueba cada λ y se elige el de menor error



Para cada λ se ajusta con cuatro partes del entrenamiento y se mide en la quinta, cinco veces, y se promedia. Se elige el λ de menor error; la prueba queda intacta.

El resultado en datos que no vio



Ridge baja el error de 342 a 316, y Lasso a 325 con 7 variables. En 20 repartos, cada uno le gana a mínimos cuadrados en 17. Un reparto es otra forma de partir a los 263 en entrenamiento y prueba; los cinco grupos se vuelven a armar dentro de cada uno.

Cómo quedan los modelos finales

Los modelos finales, escritos: el salario en miles de dólares a partir de las variables estandarizadas

Lasso ($\lambda = 33,9$): quedan 7 variables; las otras 12 tienen coeficiente exactamente 0

$$\begin{aligned}\text{salario} = & 533 + 47 \cdot z(\text{hits}) + 2 \cdot z(\text{impulsadas}) + 64 \cdot z(\text{bases_bolas}) + 16 \cdot z(\text{jonrones_carrera}) \\ & + 179 \cdot z(\text{impulsadas_carrera}) + 107 \cdot z(\text{outs_defensa}) - 38 \cdot z(\text{division_oeste})\end{aligned}$$

Ridge ($\lambda = 94$): las 19 variables, todas con coeficientes chicos; las siete de la carrera se reparten el mérito

$$\begin{aligned}\text{salario} = & 533 + 3 \cdot z(\text{turnos}) + 37 \cdot z(\text{hits}) + 1 \cdot z(\text{jonrones}) + 19 \cdot z(\text{anotadas}) + 33 \cdot z(\text{impulsadas}) \\ & + 45 \cdot z(\text{bases_bolas}) + 8 \cdot z(\text{temporadas}) + 18 \cdot z(\text{turnos_carrera}) + 33 \cdot z(\text{hits_carrera}) + 42 \cdot z(\text{jonrones_carrera}) \\ & + 34 \cdot z(\text{anotadas_carrera}) + 42 \cdot z(\text{impulsadas_carrera}) + 25 \cdot z(\text{bases_bolas_carrera}) + 77 \cdot z(\text{outs_defensa}) - 2 \cdot z(\text{asistencias}) \\ & - 1 \cdot z(\text{errores}) + 8 \cdot z(\text{liga_nacional}) - 43 \cdot z(\text{division_oeste}) + 5 \cdot z(\text{liga_nacional_1987})\end{aligned}$$

$z(\text{variable})$ es la variable estandarizada: cuántas desviaciones estándar sobre la media del entrenamiento. El 533 es el salario promedio de los 131.

Cómo se lee un coeficiente: el 179 de `impulsadas_carrera` en Lasso dice que una desviación estándar más (353 impulsadas en la carrera) son 179 mil dólares más al año, con el resto igual.

Lasso deja una ecuación de 7 términos; Ridge, una de 19 con coeficientes chicos. Los coeficientes son por desviación estándar de cada variable.

Ridge y Lasso en scikit-learn

```
escala = StandardScaler().fit(X_ent)
Z_ent = escala.transform(X_ent)
Z_pru = escala.transform(X_pru)
lasso = LassoCV(cv=5).fit(Z_ent, y_ent)
lasso.predict(Z_pru)
```

- LassoCV elige λ con validación cruzada; Ridge es igual, con RidgeCV. scikit-learn llama alpha a λ .

¿Ridge o Lasso?

Ridge y Lasso: la misma idea, dos castigos, dos resultados

	Ridge	Lasso
castigo	suma de cuadrados: $\lambda \sum b^2$	suma de valores absolutos: $\lambda \sum b $
qué hace con los coeficientes	achica todos; ninguno llega a 0	achica todos y lleva algunos a 0
el modelo que deja	usa las 19 variables	usa 7: elige variables
cuándo conviene	muchas variables aportan un poco o dicen lo mismo	se sospecha que pocas variables importan; se quiere un modelo simple
error en prueba, este reparto	316 (mínimos cuadrados: 342)	325 (mínimos cuadrados: 342)

Ninguno es mejor en todos los casos (James et al., p. 224). En los dos, λ se elige con validación cruzada y el error se mide en prueba.

La respuesta a la pregunta de la clase

- ▶ **Equivocarse cuesta distinto según el error y según dónde se mida.**
- ▶ En un clasificador, la matriz separa los dos errores; sensibilidad, especificidad y precisión miden cada uno, y el umbral se elige con el costo de cada error, no por defecto en 0,5.
- ▶ En una regresión con muchas variables, el error que vale es el de prueba: mínimos cuadrados se ajusta al azar de su muestra, y Ridge o Lasso lo corrigen castigando los coeficientes grandes, con λ elegido por validación cruzada.

Síntesis

- ▶ Sensibilidad, especificidad y precisión miden un error por vez; la exactitud los mezcla.
- ▶ El umbral de menor costo es $\text{costo FP} / (\text{costo FP} + \text{costo FN})$; con costos iguales, 0,5.
- ▶ La curva ROC compara modelos con todos los umbrales; el AUC lo resume en un número.
- ▶ Con muchas variables, un mejor ajuste en entrenamiento puede ser más error en prueba.
- ▶ Ridge achica y estabiliza todos los coeficientes; Lasso lleva algunos a 0 y elige variables; λ se elige con validación cruzada.

Recreo

10 minutos · al volver: manos al teclado, con el notebook

Próxima clase

- ▶ El jueves 17 no hay clases.
- ▶ Martes 22, la última clase del módulo: reducción de dimensionalidad (PCA) y las presentaciones orales del avance de cada equipo.
- ▶ **Hoy se entrega el análisis exploratorio; el informe de avance, el viernes 25.**

Referencias

James, G., Witten, D., Hastie, T. y Tibshirani, R. (2013). An Introduction to Statistical Learning. Springer. Pp. 145 a 149 (sensibilidad, especificidad y curva ROC, con los datos Default); pp. 165 a 167 (precisión, con los datos Caravan); sección 5.1.3, p. 181 (validación cruzada); sección 6.2, pp. 214 a 228 (Ridge y Lasso); laboratorio 6.6, pp. 251 a 255 (Ridge y Lasso con los datos Hitters). PDF gratuito en statlearning.com.

Documentación de scikit-learn sobre modelos lineales, Ridge y Lasso:
scikit-learn.org/stable/modules/linear_model.html

Datos: Default y Hitters, del libro de James et al. (statlearning.com).